Understanding Method Overloading and Method Overriding in Java

---

1. Method Overloading

Definition:

Method overloading in Java occurs when two or more methods in the same class share the same name but have different parameters. The difference can be in:

- The number of parameters
- The type of parameters
- The order of parameters

It allows methods to perform similar operations but with different types or numbers of inputs.

Key Points:

1. Overloading is resolved at compile-time (also known as compile-time polymorphism).
2. It can occur within the same class.
3. Method overloading does not depend on return type alone.

Example:

```
class Calculator {
    // Overloaded method to add two integers
    int add(int a, int b) {
        return a + b;
    }

    // Overloaded method to add three integers
    int add(int a, int b, int c) {
        return a + b + c;
    }

    // Overloaded method to add two doubles
    double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();

        System.out.println("Addition of two integers: " + calc.add(5, 10));
        System.out.println("Addition of three integers: " + calc.add(5, 10,
15));
        System.out.println("Addition of two doubles: " + calc.add(5.5,
10.5));
    }
}
```

Output:

```
Addition of two integers: 15
```

```
Addition of three integers: 30
Addition of two doubles: 16.0
```


2. Method Overriding

Definition:

Method overriding occurs when a subclass provides a specific implementation of a method that is already defined in its parent class. The method in the subclass must have the same name, return type, and parameters as the method in the parent class.

Key Points:

1. Overriding is resolved at runtime (also known as runtime polymorphism).
2. It must involve inheritance (subclass and superclass).
3. The `@Override` annotation is recommended to ensure correct overriding.
4. Access modifiers of the overridden method in the subclass cannot be more restrictive than those in the superclass.
5. Static methods cannot be overridden.

Example:

```
class Parent {
    void displayMessage() {
        System.out.println("Message from Parent class.");
    }
}

class Child extends Parent {
    @Override
    void displayMessage() {
        System.out.println("Message from Child class.");
    }
}

public class Main {
    public static void main(String[] args) {
```

```
        Parent obj1 = new Parent();
        obj1.displayMessage(); // Calls Parent class method

        Parent obj2 = new Child();
        obj2.displayMessage(); // Calls Child class method
    }
}
```

Output:

```
Message from Parent class.
Message from Child class.
```

---

3. Key Differences Between Method Overloading and Method Overriding

Feature	Method Overloading	Method Overriding
Definition	Same method name, different parameters.	Same method name, same parameters in subclass.
Polymorphism Type	Compile-time polymorphism.	Runtime polymorphism.
Inheritance	Not required.	Required.
Access Modifiers	Can have any access modifier.	Cannot reduce visibility.
Static Methods	Can be overloaded.	Cannot be overridden.
Annotations	No annotation required.	Recommended to use `@Override`.

---

4. Common Interview Questions

Basic Questions:

1. What is method overloading?
2. What is method overriding?
3. Can a static method be overridden? Why or why not?
4. What is the use of the `@Override` annotation?

Scenario-Based Questions:

1. Can we overload methods by just changing the return type?
2. How is runtime polymorphism achieved in Java?
3. Explain method overriding with an example of upcasting.
4. What happens if we use a private method in a parent class and try to override it?

Code-Based Questions:

1. Write a program to demonstrate method overloading.
2. Write a program where a subclass overrides a method from the superclass and includes additional functionality.
3. Write a program to show what happens when a static method is "overridden."

---

5. Advanced Concepts

Overloading with Varargs:

```
class VarargsExample {
    void printNumbers(int... numbers) {
        for (int num : numbers) {
            System.out.print(num + " ");
        }
        System.out.println();
    }

    void printNumbers(String message, int... numbers) {
        System.out.print(message + ": ");
        for (int num : numbers) {
            System.out.print(num + " ");
        }
        System.out.println();
    }
}

public class Main {
    public static void main(String[] args) {
        VarargsExample example = new VarargsExample();

        example.printNumbers(1, 2, 3);
        example.printNumbers("Numbers", 4, 5, 6);
    }
}
```

Output:

```
1 2 3
Numbers: 4 5 6
```

Using Covariant Return Types in Overriding:

```
class Parent {
    Parent getObject() {
        return this;
    }
}

class Child extends Parent {
    @Override
    Child getObject() {
        return this;
    }
}

public class Main {
    public static void main(String[] args) {
        Parent obj = new Child();
        System.out.println(obj.getObject().getClass().getSimpleName());
    }
}
```

Output:

```
Child
```

---

6. Conclusion

Method overloading and overriding are powerful tools in Java that allow developers to implement polymorphism. While overloading focuses on compile-time behavior, overriding enables dynamic, runtime-based functionality. Mastery of these concepts is essential for clean, maintainable, and efficient code.
